

Java Reservation System

Thread Synchronization and Shared Seat Booking

1 Objective / Aim

To demonstrate Java multithreading and synchronization by sharing a reservation object among several person threads. The synchronized booking operation protects the shared seat count so concurrent requests cannot reserve the same seats.

2 System Requirements

- Hardware: Standard computer capable of running a Java development environment.
- Software: Windows, Linux, or macOS; JDK 17 or later; a text editor or IDE such as VS Code, IntelliJ IDEA, Eclipse, or NetBeans.

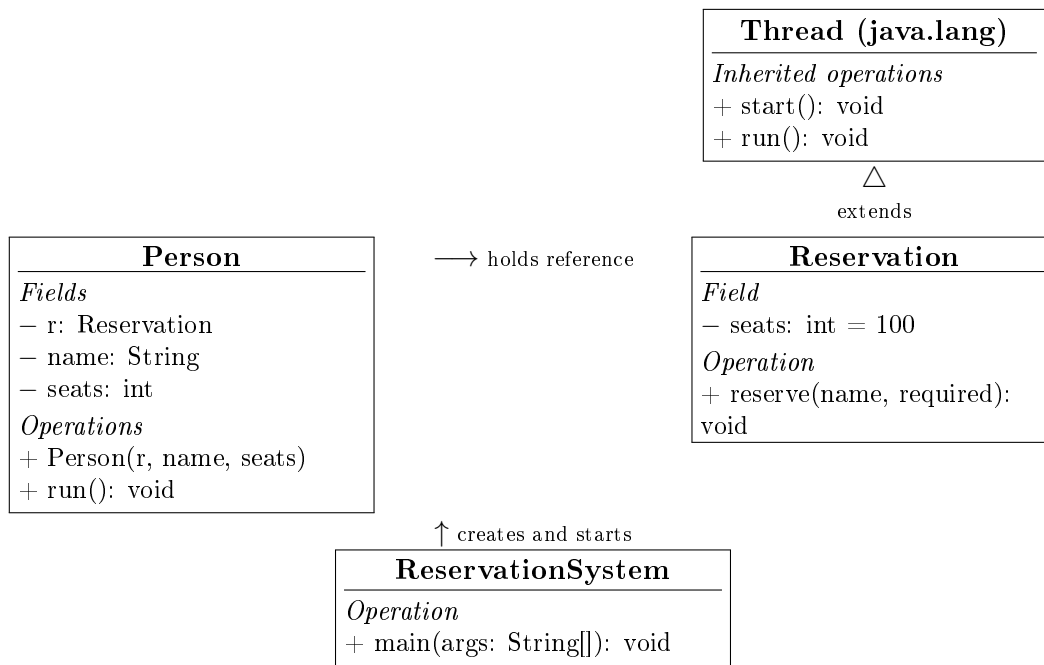
3 Algorithm / Logic Description

1. Create a **Reservation** object with 100 available seats.
2. Create four **Person** threads, each holding a reference to the same reservation object and its requested seat count.
3. Start all four threads. Each thread calls **reserve**.
4. Inside **reserve**, synchronize on the shared reservation object so only one thread checks and updates the seat count at a time.
5. If enough seats remain, subtract the requested seats and print the booking and remaining count; otherwise print that seats are unavailable.

The order in which the threads acquire the lock depends on the scheduler, so the printed booking order can vary. The synchronized block keeps each availability check and update atomic.

4 UML Class Diagram

The UML boxes show each class's important fields and operations. **Person** extends Java's **Thread**; each person holds a reservation reference. The main class creates the shared reservation and person threads.



UML visibility: + public, - private. The association from **Person** to **Reservation** represents its **r** field; the inheritance arrow marks the superclass.

5 Source Code

Save this source as `ReservationSystem.java`. The code listing is shown without line numbers or a surrounding border.

```
class Reservation {

    private int seats = 100;

    void reserve(String name, int required) {

        synchronized (this) {

            System.out.println(name +
                               " requested " + required + " seats.");

            if (required <= seats) {

                System.out.println(name + " is booking...");

                seats -= required;

                System.out.println(name +
                                   " booked " + required + " seats.");

                System.out.println("Remaining seats: " + seats);

            } else {

                System.out.println(name +
                                   " - Seats not available.");

            }

            System.out.println();

        }

    }

}

class Person extends Thread {

    Reservation r;
    String name;
    int seats;

    Person(Reservation r, String name, int seats) {

        this.r = r;
        this.name = name;
        this.seats = seats;

    }

    public void run() {

        r.reserve(name, seats);

    }

}

public class ReservationSystem {

    public static void main(String[] args) {

        Reservation r = new Reservation();

        Person p1 = new Person(r, "Person A", 35);
        Person p2 = new Person(r, "Person B", 25);
        Person p3 = new Person(r, "Person C", 30);
        Person p4 = new Person(r, "Person D", 20);

    }

}
```

```
        p1.start();  
        p2.start();  
        p3.start();  
        p4.start();  
    }  
}
```

6 Compilation and Execution

Open a terminal in the folder containing the source file and run:

```
javac ReservationSystem.java
java ReservationSystem
```

7 Sample Output

Thread scheduling can change the order of the requests. One possible run is:

```
Person A requested 35 seats.
Person A is booking...
Person A booked 35 seats.
Remaining seats: 65
```

```
Person B requested 25 seats.
Person B is booking...
Person B booked 25 seats.
Remaining seats: 40
```

```
Person C requested 30 seats.
Person C is booking...
Person C booked 30 seats.
Remaining seats: 10
```

```
Person D requested 20 seats.
Person D - Seats not available.
```

8 Result

The program demonstrates thread creation with **Thread** and uses a synchronized block to coordinate simultaneous reservation requests safely. With requests totaling 110 seats and only 100 seats available, one request is denied.